# B M S COLLEGE OF ENGINEERING

*(An Autonomous Institution Affiliated to VTU, Belagavi)*

**Post Box No.: 1908, Bull Temple Road, Bengaluru – 560 019**

# DEPARTMENT OF MACHINE LEARNING

**Academic Year: 2021-2022 (Session: Oct2021-April 2022)**

# DATA STRUCTURES (20AM3PCDST)

**ALTERNATIVE ASSESSMENT TOOL (AAT)**

## PRODUCER CONSUMER PROBLEM IN OPERATING SYSTEM

**Submitted by**

| | |
|---|---|
| Student Name: | **Siddarth B Ambannavar** |
| USN: | **1BM20AI049** |
| Date: | **19-1-2022** |
| Semester & Section: | **3A** |
| Total Pages: | **23** |
| Student Signature: | **Siddarth.B.A** |

**Valuation Report (to be filled by the faculty)**

| | |
|---|---|
| Score: | |
| Comments: | |
| Faculty In-charge: | **Dr. Monika Puttaramaiah** |
| Faculty Signature: with date | |

1. **About the Application ( Write a brief note on the application, its working and data structure concept employed to store and retrieve the data)**

Producer-consumer problem is a common problem in OS where, when a process is producing the data, at the same time another process uses or receives the same data. This can sometimes create problems such as race condition. So to avoid this we make use of Buffer which is implemented using Circular Queue data structure. When the Buffer is full we don't allow the producer process to send the data. At the same time when the buffer is empty we restrict the consumer process from receiving the data. In buffer we store the data in queue fashion. The data which comes first will the first one to be received by consumer.

2. **Working Code**

```
// PRODUCER CONSUMER PROBLEM USING BUFFER, CIRCULAR QUEUE DATA STRUCTURE

#include <iostream>
using namespace std;
#define BUFFER_SIZE 5

// BUFFER structure
struct BUFFER
{
	int front;
	int rear;
	int size;
	int capacity;
	int *arr;
};
```

```
// CONSUMER structure
struct CONSUMER
{
	int *arr;
	int size;
	int idx;
};

typedef struct BUFFER *Buffer;
typedef struct CONSUMER *Consumer;

// New Buffer
Buffer createBuffer()
{
	Buffer buffer = new BUFFER;
	buffer->capacity = BUFFER_SIZE;
	buffer->size = 0;
	buffer->front = 0;
	buffer->rear = BUFFER_SIZE - 1;
	buffer->arr = (int *)malloc(buffer->capacity * sizeof(int));
	return buffer;
}

// Consumer Array
Consumer createConsumer()
{
```

```
        Consumer consumer = new CONSUMER;
        consumer->size = BUFFER_SIZE;
        consumer->idx = -1;
        consumer->arr = (int *)malloc(BUFFER_SIZE * sizeof(int));
        return consumer;
}

// Check if BUFFER is Full
bool isFullBuffer(Buffer buffer)
{
        return (buffer->size == buffer->capacity);
}

// Check if CONSUMER is Full
bool isFullConsumer(Consumer consumer)
{
        return (consumer->idx == consumer->size - 1);
}

// Check if BUFFER is Empty
bool isEmptyBuffer(Buffer buffer)
{
        return (buffer->size == 0);
}

// Check if CONSUMER is Empty
bool isEmptyConsumer(Consumer consumer)
```

```
{
	return (consumer->size == 0);
}

// Function to enable the PRODUCER to add the item to buffer iff buffer is not full
void addItem(Buffer buffer, int item)
{
	if (isFullBuffer(buffer))
	{
		cout << "Buffer is full. Wait for the consumer to receive the item." << endl;
		return;
	}
	cout << "Enter the item to be added: ";
	cin >> item;
	buffer->rear = (buffer->rear + 1) % buffer->capacity;
	buffer->arr[buffer->rear] = item;
	buffer->size++;
}

// Function for CONSUMER to remove the item from buffer iff buffer is not empty
int removeItem(Buffer buffer, Consumer consumer)
{
	if (isEmptyBuffer(buffer))
	{
```

```
		cout << "Buffer is empty. Wait for the producer to add an item." << endl;
		return -1;
	}
	int item = buffer->arr[buffer->front];
	buffer->front = (buffer->front + 1) % buffer->capacity;
	buffer->size--;
	if (isFullConsumer(consumer))
	{
		consumer->arr = (int *)realloc(consumer->arr, (++consumer->size) * sizeof(int));
	}
	consumer->arr[++consumer->idx] = item;
	return item;
}

// Function to display the items in buffer
void displayBuffer(Buffer buffer)
{
	if (isEmptyBuffer(buffer))
	{
		cout << "Buffer is empty" << endl;
		return;
	}
	int i = buffer->front;
	while (i != buffer->rear)
	{
```

```
            cout << buffer->arr[i] << " ";
            i = (i + 1) % buffer->capacity;
        }
        cout << buffer->arr[i] << endl;
}

// Function to display the contents already recieved by the consumer
void displayConsumer(Consumer consumer)
{
        if (isEmptyConsumer(consumer))
        {
            cout << "Consumer is empty" << endl;
            return;
        }
        for (int i = 0; i <= consumer->idx; i++)
        {
            cout << consumer->arr[i] << " ";
        }
        cout << endl;
}

void displayIntro()
{
        printf("\n------PRODUCER-CONSUMER------\n");
        printf("Press 1 for producer to add an item into the buffer\n");
        printf("Press 2 for consumer to remove an item from the buffer\n");
        printf("Press 3 to display the buffer\n");
```

```
	printf("Press 4 to display the items already recieved by the consumer\n");
	printf("Press 5 to exit\n");
	printf("----------------------------------------------\n");
}

int main()
{
	displayIntro();

	Buffer buffer = createBuffer();
	Consumer consumer = createConsumer();

	int choice, item;
	while (1)
	{
	f:
		cout << "\nEnter your choice: ";
		cin >> choice;
		cout << endl;

		switch (choice)
		{
		case 1:
			cout << "Producer is trying to add an item into the buffer" << endl;
			addItem(buffer, item);
```

```
                cout << "BUFFER: ";
                displayBuffer(buffer);
                cout << endl;
                break;

        case 2:
                cout << "Consumer is trying to remove an item from the buffer" << endl;
                item = removeItem(buffer, consumer);
                if (item == -1)
                        cout << "Removed item is: NULL" << endl;
                else
                        cout << "Removed item is: " << item << endl;
                cout << "BUFFER: ";
                displayBuffer(buffer);
                cout << "CONSUMER: ";
                displayConsumer(consumer);
                cout << endl;
                break;

        case 3:
                cout << "BUFFER: ";
                displayBuffer(buffer);
                cout << endl;
                break;

        case 4:
```

```
                cout << "CONSUMER: ";
                displayConsumer(consumer);
                cout << endl;
                break;

            case 5:
                cout << "Exiting..." << endl;
                exit(0);

            default:
                cout << "Invalid choice" << endl;
                goto f;
            }
        }

        return 0;
}
```

3. **Screen shots**

```
// * PRODUCER CONSUMER PROBLEM USING BUFFER, CIRCULAR QUEUE DATA STRUCTURE

#include <iostream>
using namespace std;
#define BUFFER_SIZE 5

// * Buffer structure
struct BUFFER
{
    int front;
    int rear;
    int size;
    int capacity;
    int *arr;
};

// * Consumer structure
struct CONSUMER
{
    int *arr;
    int size;
    int idx;
};

typedef struct BUFFER *Buffer;
typedef struct CONSUMER *Consumer;

// * New Buffer
Buffer createBuffer()
{
    Buffer buffer = new BUFFER;
    buffer->capacity = BUFFER_SIZE;
    buffer->size = 0;
    buffer->front = 0;
    buffer->rear = BUFFER_SIZE - 1;
    buffer->arr = (int *)malloc(buffer->capacity * sizeof(int));
    return buffer;
}

// * Consumer Array
Consumer createConsumer()
{
    Consumer consumer = new CONSUMER;
    consumer->size = BUFFER_SIZE;
    consumer->idx = -1;
    consumer->arr = (int *)malloc(BUFFER_SIZE * sizeof(int));
    return consumer;
}
```

```
// * Check if BUFFER is Full
bool isFullBuffer(Buffer buffer)
{
    return (buffer->size == buffer->capacity);
}

// * Check if CONSUMER is Full
bool isFullConsumer(Consumer consumer)
{
    return (consumer->idx == consumer->size - 1);
}

// * Check if BUFFER is Empty
bool isEmptyBuffer(Buffer buffer)
{
    return (buffer->size == 0);
}

// * Check if CONSUMER is Empty
bool isEmptyConsumer(Consumer consumer)
{
    return (consumer->size == 0);
}
```

```
// * Function to enable the PRODUCER to add the item to buffer iff buffer is not full
void addItem(Buffer buffer, int item)
{
    if (isFullBuffer(buffer))
    {
        cout << "Buffer is full. Wait for the consumer to receive the item." << endl;
        return;
    }
    cout << "Enter the item to be added: ";
    cin >> item;
    buffer->rear = (buffer->rear + 1) % buffer->capacity;
    buffer->arr[buffer->rear] = item;
    buffer->size++;
}
```

```
// * Function for CONSUMER to remove the item from buffer iff buffer is not empty
int removeItem(Buffer buffer, Consumer consumer)
{
    if (isEmptyBuffer(buffer))
    {
        cout << "Buffer is empty. Wait for the producer to add an item." << endl;
        return -1;
    }
    int item = buffer->arr[buffer->front];
    buffer->front = (buffer->front + 1) % buffer->capacity;
    buffer->size--;
    if (isFullConsumer(consumer))
    {
        consumer->arr = (int *)realloc(consumer->arr, (++consumer->size) * sizeof(int));
    }
    consumer->arr[++consumer->idx] = item;
    return item;
}
```

```
// * Function to display the items in buffer
void displayBuffer(Buffer buffer)
{
    if (isEmptyBuffer(buffer))
    {
        cout << "Buffer is empty" << endl;
        return;
    }
    int i = buffer->front;
    while (i != buffer->rear)
    {
        cout << buffer->arr[i] << " ";
        i = (i + 1) % buffer->capacity;
    }
    cout << buffer->arr[i] << endl;
}
```

```
// * Function to display the contents already recieved by the consumer
void displayConsumer(Consumer consumer)
{
    if (isEmptyConsumer(consumer))
    {
        cout << "Consumer is empty" << endl;
        return;
    }
    for (int i = 0; i <= consumer->idx; i++)
    {
        cout << consumer->arr[i] << " ";
    }
    cout << endl;
}

void displayIntro()
{
    printf("\n------PRODUCER-CONSUMER------\n");
    printf("Press 1 for producer to add an item into the buffer\n");
    printf("Press 2 for consumer to remove an item from the buffer\n");
    printf("Press 3 to display the buffer\n");
    printf("Press 4 to display the items already recieved by the consumer\n");
    printf("Press 5 to exit\n");
    printf("----------------------------------------------\n");
}
```

```
int main()
{
    displayIntro();

    Buffer buffer = createBuffer();
    Consumer consumer = createConsumer();

    int choice, item;
    while (1)
    {
    f:
        cout << "\nEnter your choice: ";
        cin >> choice;
        cout << endl;

        switch (choice)
        {
        case 1:
            cout << "Producer is trying to add an item into the buffer" << endl;
            addItem(buffer, item);
            cout << "BUFFER: ";
            displayBuffer(buffer);
            cout << endl;
            break;
```

```
        case 2:
            cout << "Consumer is trying to remove an item from the buffer" << endl;
            item = removeItem(buffer, consumer);
            if (item == -1)
                cout << "Removed item is: NULL" << endl;
            else
                cout << "Removed item is: " << item << endl;
            cout << "BUFFER: ";
            displayBuffer(buffer);
            cout << "CONSUMER: ";
            displayConsumer(consumer);
            cout << endl;
            break;

        case 3:
            cout << "BUFFER: ";
            displayBuffer(buffer);
            cout << endl;
            break;

        case 4:
            cout << "CONSUMER: ";
            displayConsumer(consumer);
            cout << endl;
            break;
```

```
        case 5:
            cout << "Exiting..." << endl;
            exit(0);

        default:
            cout << "Invalid choice" << endl;
            goto f;
        }
    }

    return 0;
}
```

**Output:**

```
------PRODUCER-CONSUMER------
Press 1 for producer to add an item into the buffer
Press 2 for consumer to remove an item from the buffer
Press 3 to display the buffer
Press 4 to display the items already recieved by the consumer
Press 5 to exit
----------------------------------------------

Enter your choice: 1

Producer is trying to add an item into the buffer
Enter the item to be added: 10
BUFFER: 10



Enter your choice: 1

Producer is trying to add an item into the buffer
Enter the item to be added: 20
BUFFER: 10 20
```

```
Enter your choice: 1

Producer is trying to add an item into the buffer
Enter the item to be added: 50
BUFFER: 20 30 40 50



Enter your choice: 1

Producer is trying to add an item into the buffer
Enter the item to be added: 60
BUFFER: 20 30 40 50 60



Enter your choice: 2

Consumer is trying to remove an item from the buffer
Removed item is: 20
BUFFER: 30 40 50 60
CONSUMER: 10 20
```

```
Enter your choice: 1

Producer is trying to add an item into the buffer
Enter the item to be added: 70
BUFFER: 30 40 50 60 70



Enter your choice: 1

Producer is trying to add an item into the buffer
Buffer is full. Wait for the consumer to receive the item.
BUFFER: 30 40 50 60 70



Enter your choice: 80

Invalid choice

Enter your choice: 2
```

```
Enter your choice: 1

Producer is trying to add an item into the buffer
Enter the item to be added: 30
BUFFER: 10 20 30



Enter your choice: 2

Consumer is trying to remove an item from the buffer
Removed item is: 10
BUFFER: 20 30
CONSUMER: 10



Enter your choice: 1

Producer is trying to add an item into the buffer
Enter the item to be added: 40
BUFFER: 20 30 40
```

```
Enter your choice: 2

Consumer is trying to remove an item from the buffer
Removed item is: 30
BUFFER: 40 50 60 70
CONSUMER: 10 20 30


Enter your choice: 2

Consumer is trying to remove an item from the buffer
Removed item is: 40
BUFFER: 50 60 70
CONSUMER: 10 20 30 40


Enter your choice: 2

Consumer is trying to remove an item from the buffer
Removed item is: 50
BUFFER: 60 70
```

```
Enter your choice: 2

Consumer is trying to remove an item from the buffer
Removed item is: 50
BUFFER: 60 70
CONSUMER: 10 20 30 40 50


Enter your choice: 2

Consumer is trying to remove an item from the buffer
Removed item is: 60
BUFFER: 70
CONSUMER: 10 20 30 40 50 60


Enter your choice: 3

BUFFER: 70
```

```
Enter your choice: 4

CONSUMER: 10 20 30 40 50 60



Enter your choice: 2

Consumer is trying to remove an item from the buffer
Removed item is: 70
BUFFER: Buffer is empty
CONSUMER: 10 20 30 40 50 60 70



Enter your choice: 3

BUFFER: Buffer is empty
```

```
Enter your choice: 4

CONSUMER: 10 20 30 40 50 60 70



Enter your choice: 5

Exiting...
```